Project - High Level Design
On
Project Title
CI/CD for Vue.js Technology App

Course Name: DevOps

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	AYUSHI UPADHYAY	EN22CS301258
2	AVANISH BAIRAGI	EN22CS301241
3	BHAVESH KHATRI	EN22CS301262
4	BHAVESH SHADIJA	EN22CS301264

Group Name: Placed BtechGroup06

Project Number: DO-49

Industry Mentor Name: Prof Suraj Nayak

University Mentor Name: Prof. Shyam Patel

Academic Year: 2025-26

Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. Data Design.
   3.1. Data Model
   3.2. Data Access Mechanism
   3.3. Data Retention Policies
   3.4. Data Migration

4. Interfaces

5. State and Session Management

6. Caching

7. Non-Functional Requirements
   7.1. Security Aspects
   7.2. Performance Aspects

8. References

1. Introduction

The **CI/CD Pipeline for Vue.js Application** is a fully automated software delivery system designed to streamline the development, testing, building, and deployment processes. The pipeline leverages modern DevOps tools such as **GitHub Actions or Jenkins** to ensure continuous integration and continuous delivery.

This system automates unit testing on every code commit and generates containerized artifacts upon merging into the production branch. It ensures high code quality, faster release cycles, and reliable deployments in enterprise environments.

1.1 Scope of the Document

This document provides a high-level architectural overview of the CI/CD pipeline system. It covers pipeline design, workflow automation, system components, integrations, and non-functional requirements such as scalability and security.

1.2 Intended Audience

This document is intended for:

- DevOps Engineers
- Software Developers
- System Architects
- QA/Test Engineers
- Project Stakeholders

1.3 System Overview

The system consists of multiple integrated components:

- **Source Code Repository (GitHub)** – Stores application code
- **CI Engine (GitHub Actions/Jenkins)** – Automates build and testing
- **Testing Module** – Executes unit tests
- **Build System** – Compiles Vue.js application
- **Containerization Module (Docker)** – Creates deployable artifacts
- **Container Registry (Docker Hub/AWS ECR)** – Stores images
- **Deployment Environment** – Hosts the application

The pipeline ensures seamless integration from development to production.

2. System Design

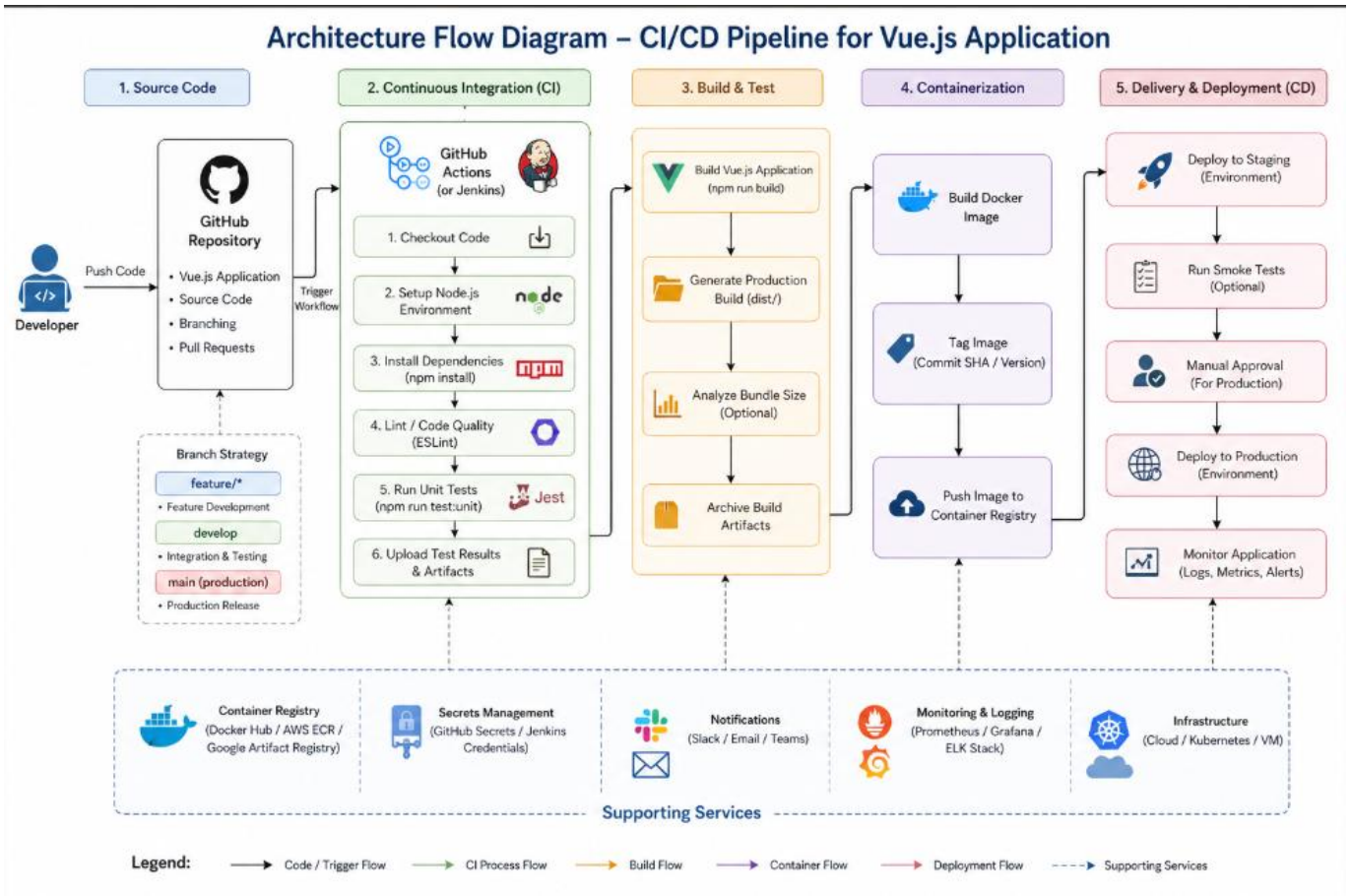
The system follows a **pipeline-based and layered architecture** ensuring modularity, scalability, and automation.

2.1 Application Design

The application is designed using a **multi-tier architecture**:

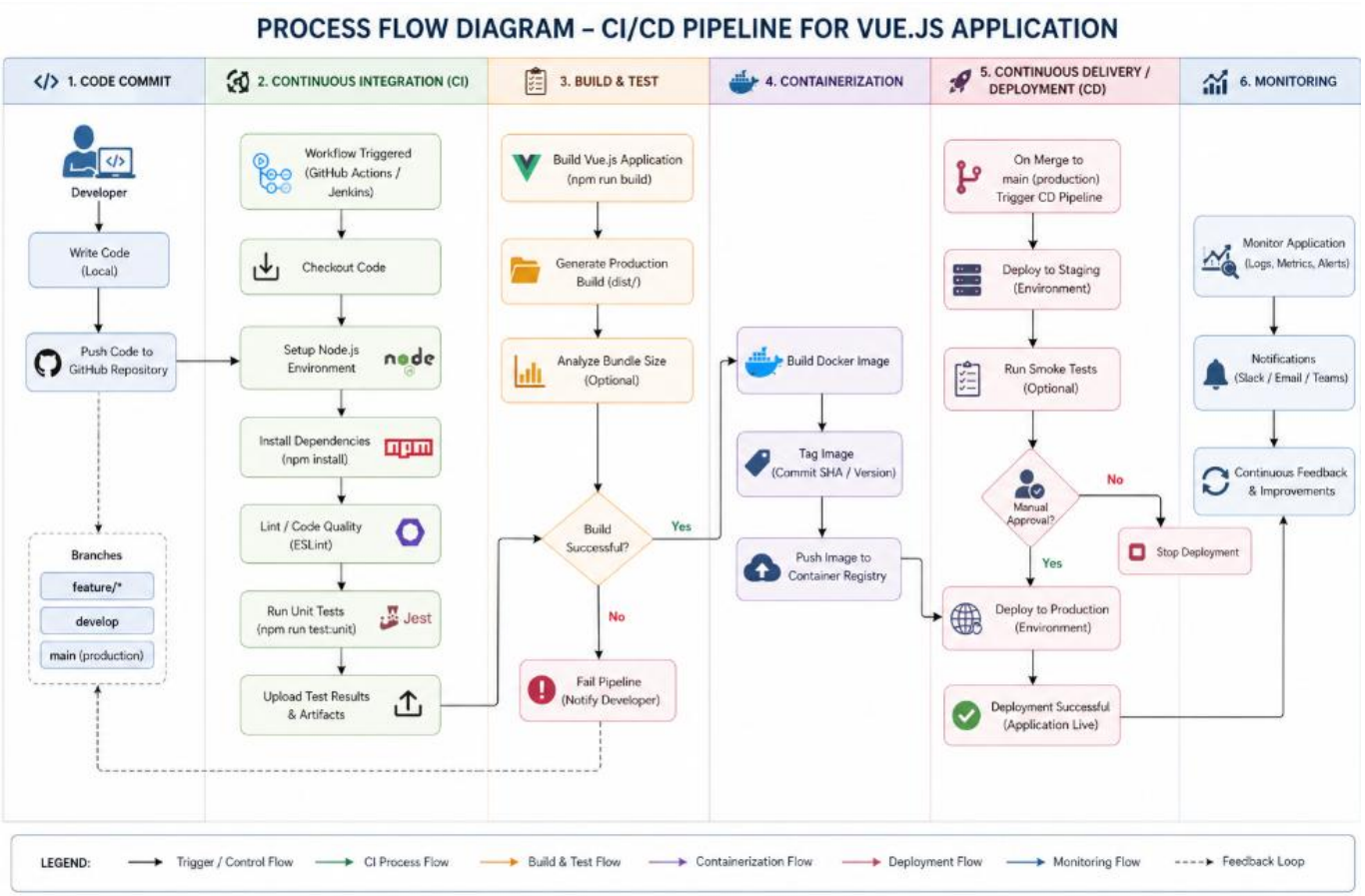
- **Frontend Layer** – Vue.js application
- **CI/CD Layer** – GitHub Actions/Jenkins workflows
- **Build & Test Layer** – Handles compilation and testing
- **Containerization Layer** – Docker image creation
- **Deployment Layer** – Server/Kubernetes deployment

This separation improves maintainability and scalability.



2.2 Process Flow

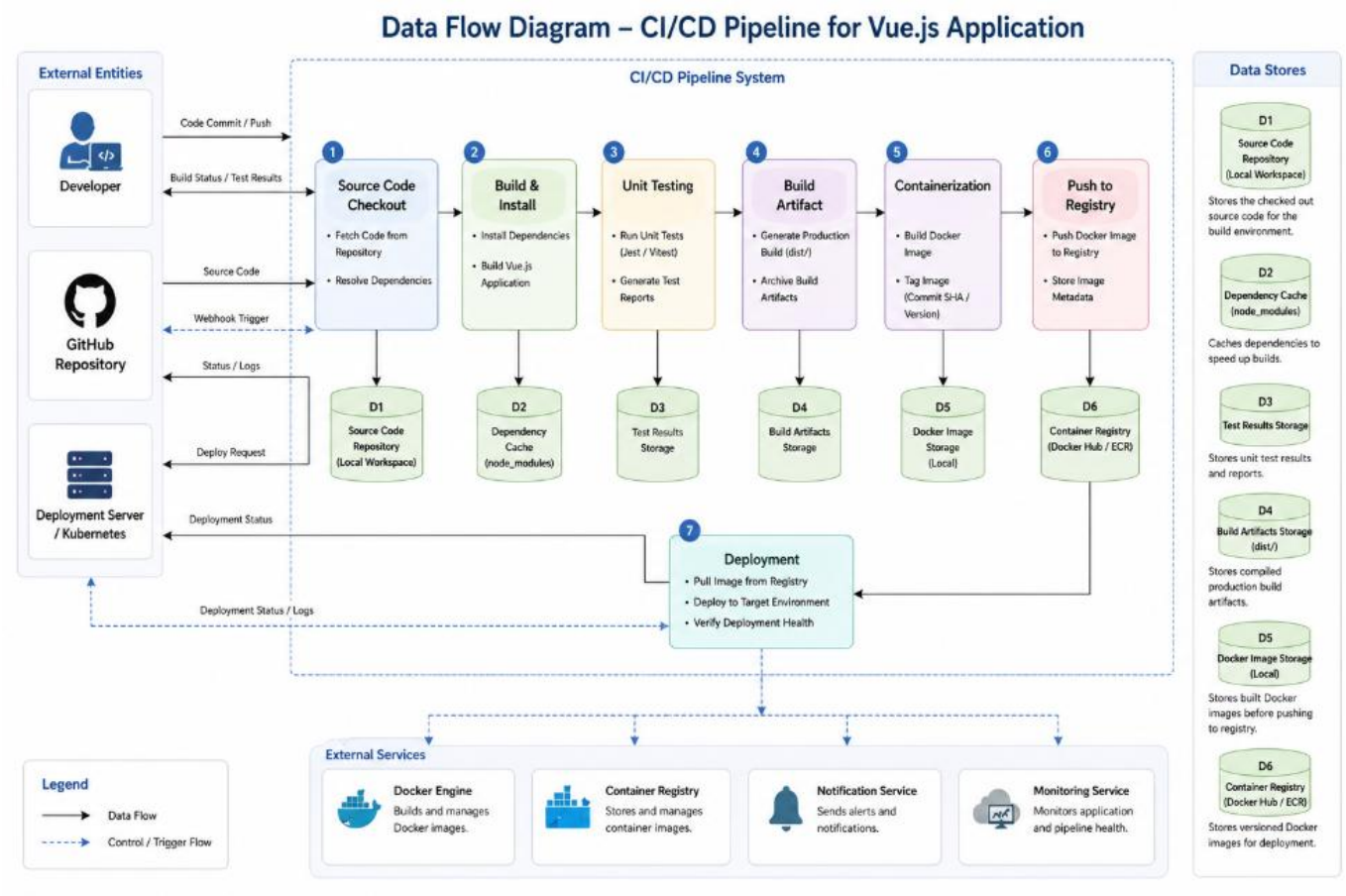
- Developer pushes code to repository
- CI pipeline is triggered automatically
- Dependencies are installed
- Unit tests are executed
- Application build is generated
- Docker image is created
- On merge to main branch → image is pushed to registry
- Deployment pipeline deploys application to server



2.3 Information Flow

- Developer → GitHub Repository → CI Pipeline
- CI Pipeline → Test Results → Build Output
- Build Output → Docker Image → Container Registry
- Registry → Deployment System → Production Environment

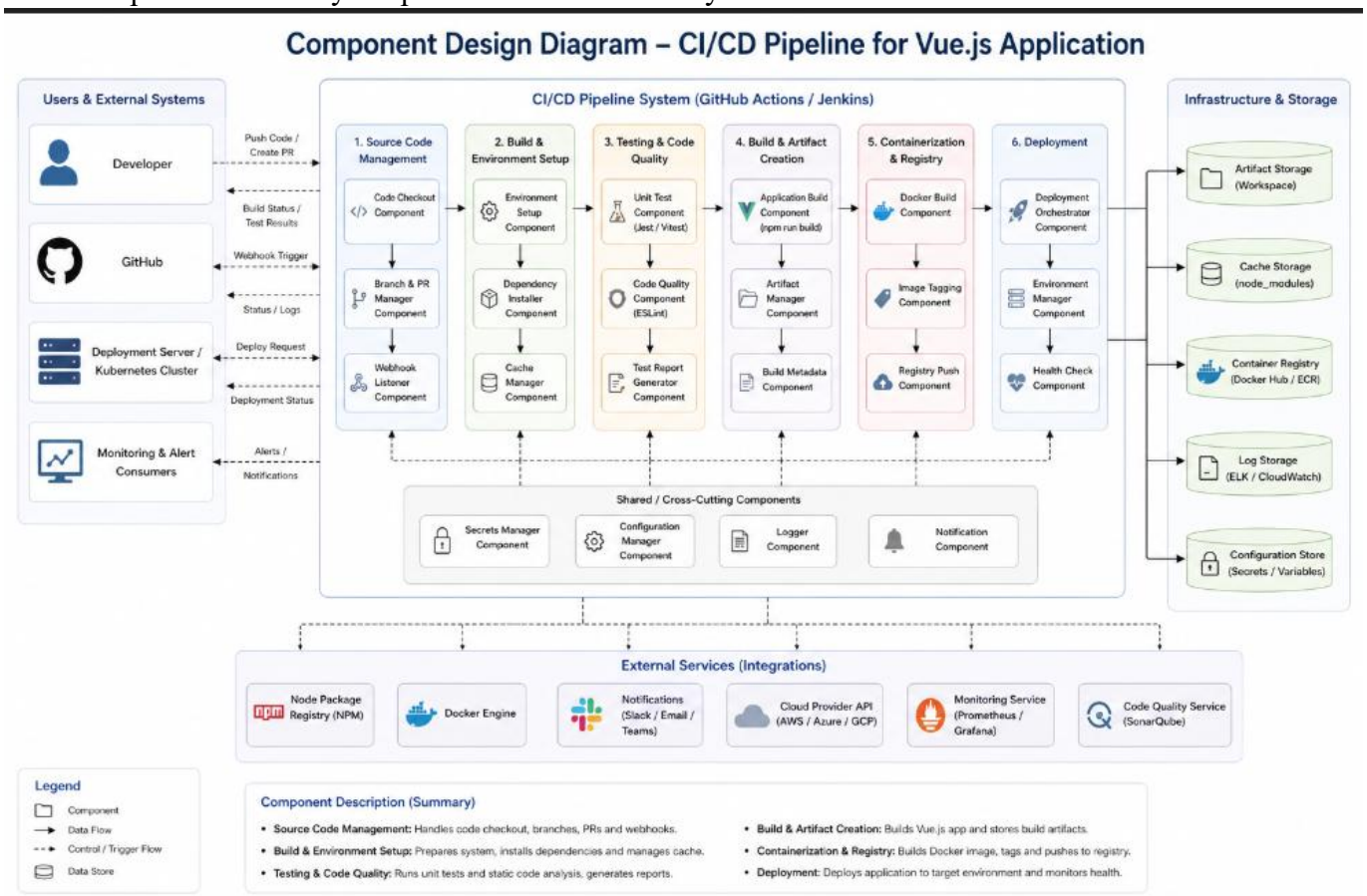
This ensures smooth and automated data flow across the pipeline.



2.4 Components Design

The system consists of the following key components:
- **Version Control System (GitHub)** – Manages codebase
- **CI/CD Engine (GitHub Actions/Jenkins)** – Executes workflows
- **Build Module** – Compiles Vue.js code
- **Testing Module** – Runs unit tests
- **Docker Engine** – Builds container images
- **Container Registry** – Stores images
- **Deployment Module** – Deploys application

Each component is loosely coupled for better scalability.



2.5 Key Design Considerations

• **Scalability** – Handle multiple builds and deployments
• **Reliability** – Ensure stable pipeline execution
• **Automation** – Minimize manual intervention
• **Security** – Protect credentials and artifacts
• **Performance Optimization** – Faster builds and deployments

2.6 API Catalogue

• **GitHub API** – Repository and workflow triggers
• **Docker API** – Image build and push
• **CI/CD API** – Pipeline execution
• **Deployment API** – Deployment automation

All APIs follow REST principles.

3. Data Design

3.1 Data Model

• **Source Code Entity** – Application code
• **Build Artifact Entity** – Compiled files
• **Docker Image Entity** – Containerized application
• **Pipeline Logs Entity** – Execution logs
• **Deployment Records Entity** – Deployment history

3.2 Data Access Mechanism

• Git-based version control
• API-based integrations
• Secure access via tokens

3.3 Data Retention Policies

• Logs retained for debugging
• Artifacts stored for versioning
• Old builds cleaned periodically

3.4 Data Migration

• Version-controlled updates
• Backup and restore mechanisms
• Artifact migration support

4. Interfaces

• **Developer Interface** – GitHub repository
• **CI/CD Interface** – GitHub Actions/Jenkins dashboard
• **Deployment Interface** – Server/Kubernetes dashboard

5. State and Session Management
• Stateless pipeline execution
• Token-based authentication
• Session logs maintained for tracking

6. Caching
• Dependency caching (node_modules)
• Docker layer caching
• Improves build speed

7. Non-Functional Requirements
7.1 Security Aspects
• Secure storage of secrets (GitHub Secrets)
• Encrypted communication (HTTPS)
• Role-based access control
• Protection against unauthorized access

7.2 Performance Aspects
• Parallel job execution
• Efficient build pipelines
• Load balancing
• Fast deployment cycles

8. References
• DevOps best practices
• GitHub Actions documentation
• Jenkins pipeline documentation
• Docker documentation
• CI/CD design principles